Trabalho relacionado

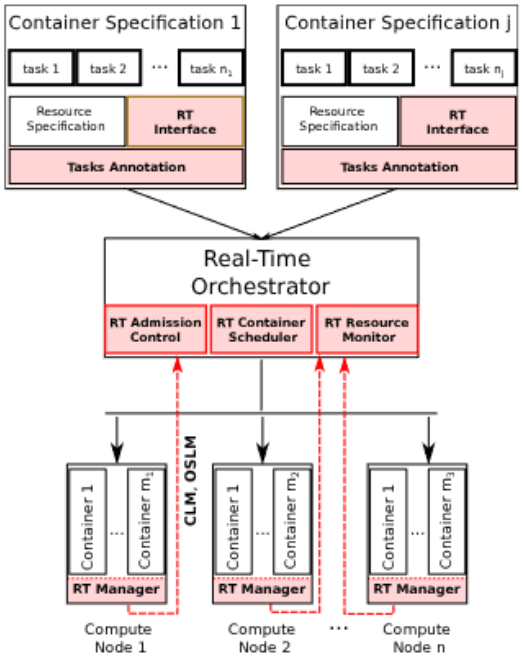
Orquestração de containers com tempo real e TSN

01

Orquestração com base no CPU

Objetivo: pôr o maior número possível de tarefas RT a correr sem que interfiram umas com as outras.

REACT



Arquitetura REACT

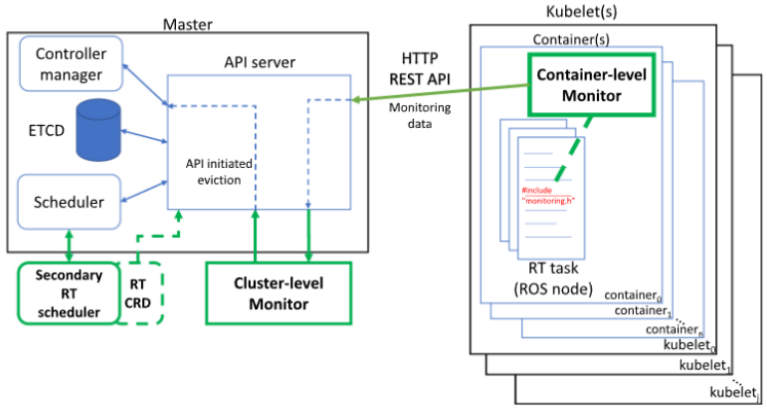
Cada container RT declara (Q, P): Q é o tempo de CPU de que precisa e P o período em que é executado.

Admissão: o nó só aceita se Σ Q/P (com o novo) ≤ 1 − δ, sendo δ a margem para overhead. Dos nós que passam, escolhe o que tem mais CPU por reservar.

No nó: recebe Q de CPU em cada período P. Se esgotar Q sem acabar, fica suspenso até ao período seguinte.

Monitoriza o nó (interrupções, trocas de contexto, I/O) e o container (deadlines falhados, lateness, tempo de resposta).

RT-Kube



Arquitetura RT-Kube

CRD com criticidade, período, deadline e WCET (Worst Case Execution Time).

Scheduler secundário: ordena por criticidade, testa Σ WCET/P $\leq$ M $\times$ 0,95 (M = nº de cores) e coloca nos nós segundo a criticidade.

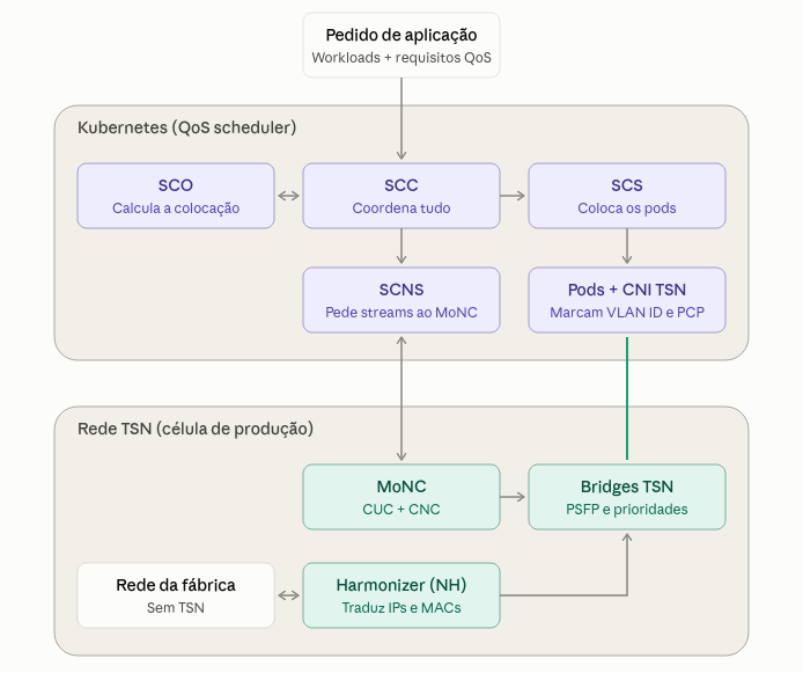
Em runtime conta deadlines falhados. Acima de um limiar, expulsa um container stateless de criticidade inferior, que é reagendado noutro nó.

02

Orquestração consciente da rede

São os mais semelhantes ao meu trabalho: propõem arquiteturas que juntam a plataforma de containers e a rede, em contexto industrial.

FabOS

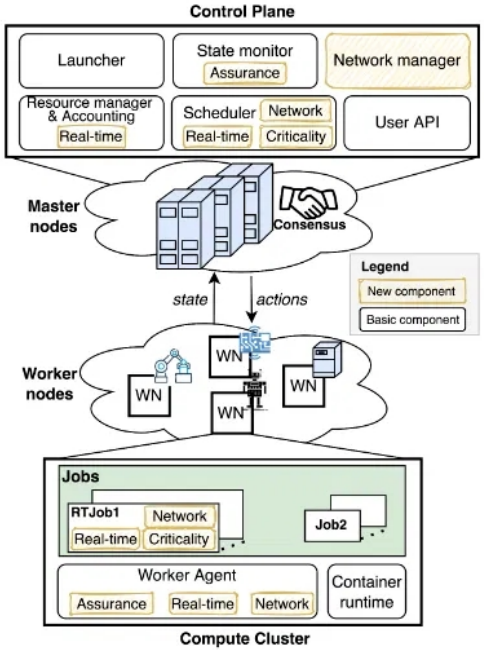


Arquitetura FabOS

1. O SCC recebe o pedido da aplicação e pede ao SCO um plano de colocação.
2. O SCNS pede ao MoNC (o CNC) os streams TSN que esse plano exige.
3. O MoNC verifica se garante as latências: se sim, configura as bridges; se não, o SCC pede novo plano.
4. Com os streams garantidos, o SCS arranca os pods. O CNI marca o VLAN ID e o PCP.

Lacunas sem monitorização em runtime; shaping por prioridade + PSFP (não TAS); não trata o RT do CPU; sem criticidade.

k4.0s

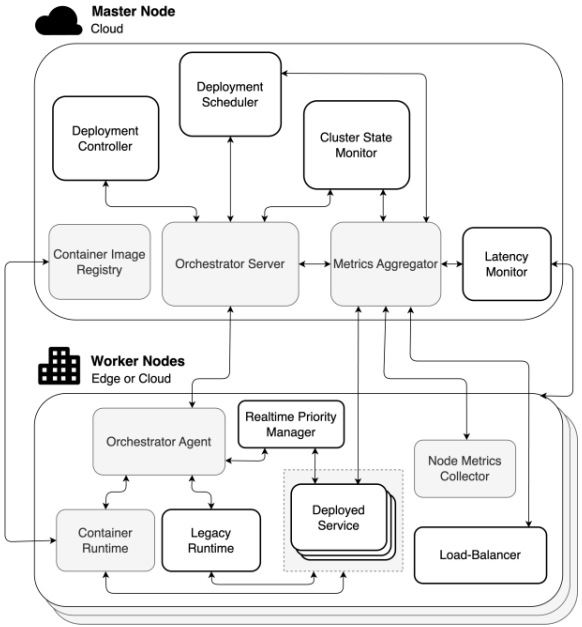


Arquitetura k4.0s

1. O utilizador submete o job pela User API, com a criticidade, períodos e WCETs, e um requisito de timeslot TDMA.
2. O Scheduler filtra os nós sem garantia suficiente, com base na informação do State monitor.
3. Pede a cada nó candidato que corra o teste de escalonabilidade, através do módulo Real-time do Worker Agent.
4. Pede ao Network manager um timeslot livre.
5. Escolhe o nó e o Launcher envia a ação.
6. Durante a execução, o módulo Assurance vigia o nó. Se uma carga vizinha começar a perturbar o job crítico, tenta eliminá-la. Se não conseguir, baixa a garantia do nó e o plano de controlo migra o job.

Lacunas a rede é TDMA, não TSN;

Edge-Cloud Continuum Orchestration



Arquitetura Edge-Cloud

Estende o Kubernetes para serviços críticos numa smart city. Usa um CRD próprio (FogService) com os parâmetros RT de cada serviço.

Plugin RT no scheduler: o Filter rejeita nós onde o pod ultrapassaria a quota RT (95%), o Score espalha os pods RT pelos nós, e o PostFilter expulsa pods de prioridade mais baixa se não houver espaço.

Plugin de dependências: coloca o pod perto das réplicas de que depende, usando a latência medida (ping) entre nós e métricas da aplicação.

Em runtime, simula de novo o scheduling de cada pod e, se houver um nó melhor, expulsa-o para ser recolocado.

Lacunas apesar de considerar a latência da rede, não usa TSN.

03

Orquestração em SDVs

Foi o único paper relevante que encontrei sobre escalonamento num veículo SDV.

Escalonamento determinístico em SDVs

Decide em conjunto onde processar cada tarefa e por que caminho, numa arquitetura zonal (4 ECUs zonais + 1 unidade central).

Objetivo: não a menor latência, mas a maior taxa de tarefas cumpridas dentro do prazo, o que importa mais para funções críticas. Também prefere caminhos mais fiáveis.

Lacunas assume que conhece as tarefas à partida (diz que é dinâmico, mas não explica como); não usa TSN.

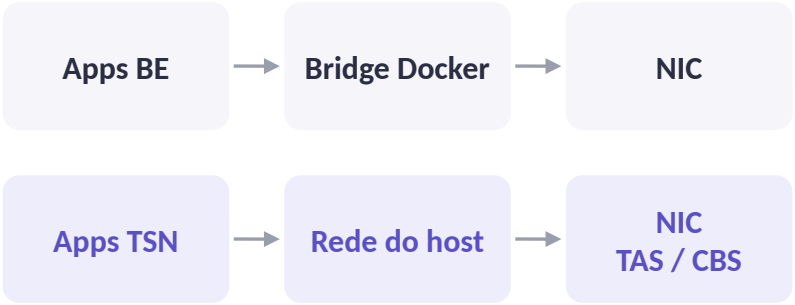
04

Extra

TSN em containers e em redes overlay.

EXTRA

Bridging TSN and Containerization



Lacuna usa a rede do host, que não é o padrão da indústria (overlay) e expõe o host.

- TSN em containers Docker: as apps BE ficam numa bridge e as apps TSN usam a rede do host, para conseguirem configurar a NIC (TAS/CBS).
- Com TAS + LaunchTime os resultados ficam quase iguais ao nativo; com CBS basta o shaper.
- Identificam dois problemas: várias apps a configurar o mesmo TAS entram em conflito, e mais containers num nó estragam configurações que eram ótimas.

EXTRA

KuberneTSN



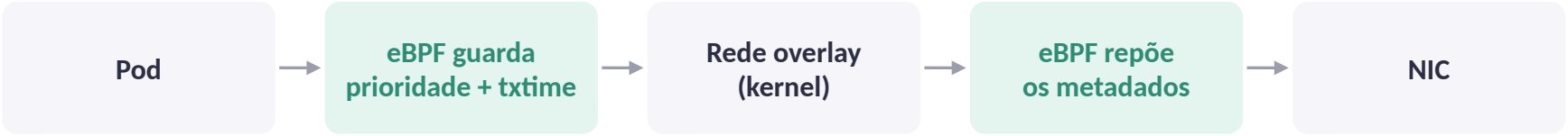
- Plugin CNI que dá TSN a pods numa rede overlay.
- Interceta os pacotes TSN antes de saírem do container. Um container próprio faz o shaping (TAS) e envia-os com kernel bypass (DPDK).
- Latência e jitter praticamente iguais a bare-metal.

Vantagem latência e jitter ao nível do bare-metal.

Desvantagens com o kernel bypass perdem-se a monitorização e a segurança do kernel. Presumo também que fique um core ocupado a fazer polling (o paper não o diz).

EXTRA

TSN Metadata Proxy



- Plugin CNI que preserva os metadados TSN nos pacotes.
- Um programa eBPF guarda a prioridade e o txtime à saída do pod e volta a pô-los antes da NIC.

Vantagem os pacotes continuam a passar pelo kernel e o custo deve ser baixo (12 bytes por pacote).

Desvantagens não reduz a latência do container e não fizeram testes de desempenho. Deixam de fora a configuração centralizada da rede.

05

Proposta